Modeling with UML

Mahesh Shirole

VJTI, Mumbai-19

Introduction

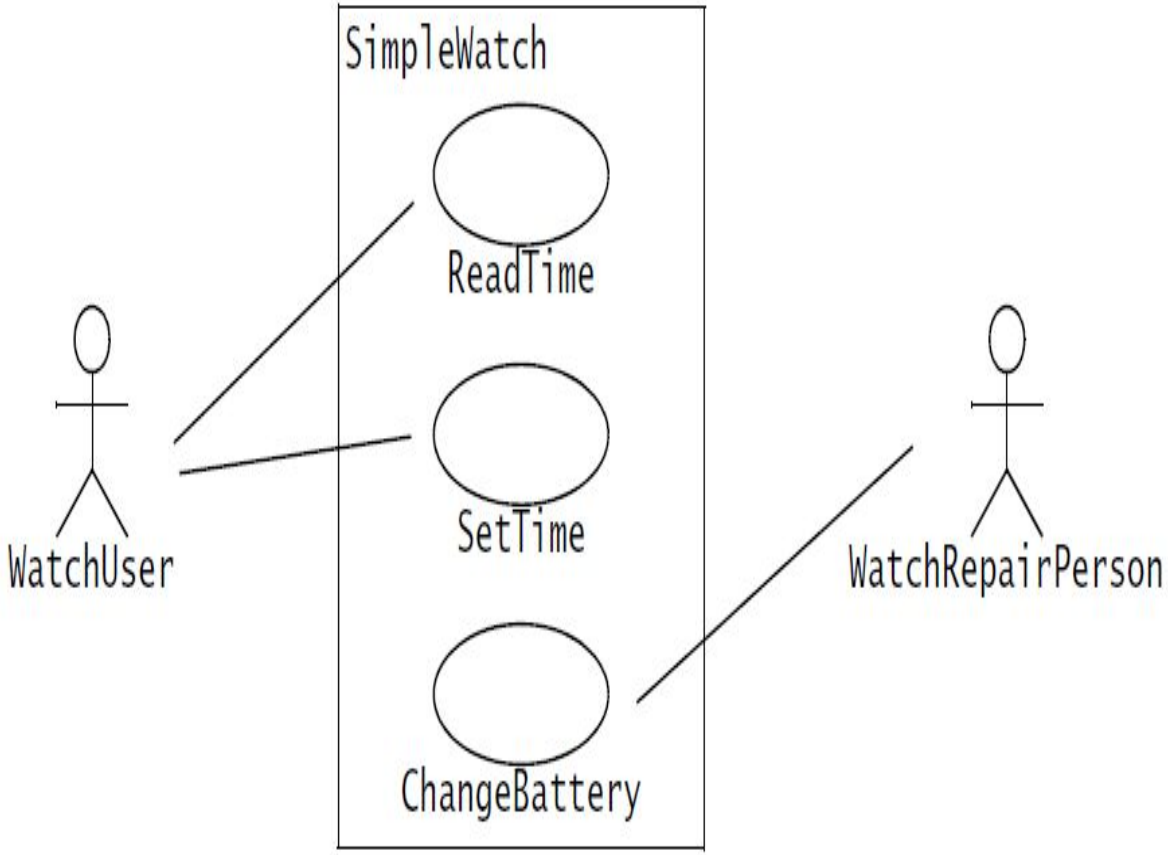
- UML is a notation that resulted from the unification of
  - OMT (Object Modeling Technique [Rumbaugh et al., 1991]),
  - Booch [Booch, 1994], and
  - OOSE (Object-Oriented Software Engineering [Jacobson et al., 1992]).
- UML has also been influenced by other object-oriented notations, such as those introduced by
  - Mellor and Shlaer [Mellor & Shlaer, 1998],
  - Coad and Yourdon [Coad et al., 1995],
  - Wirfs-Brock [Wirfs-Brock et al., 1990], and
  - Martin and Odell [Martin & Odell, 1992].

Introduction

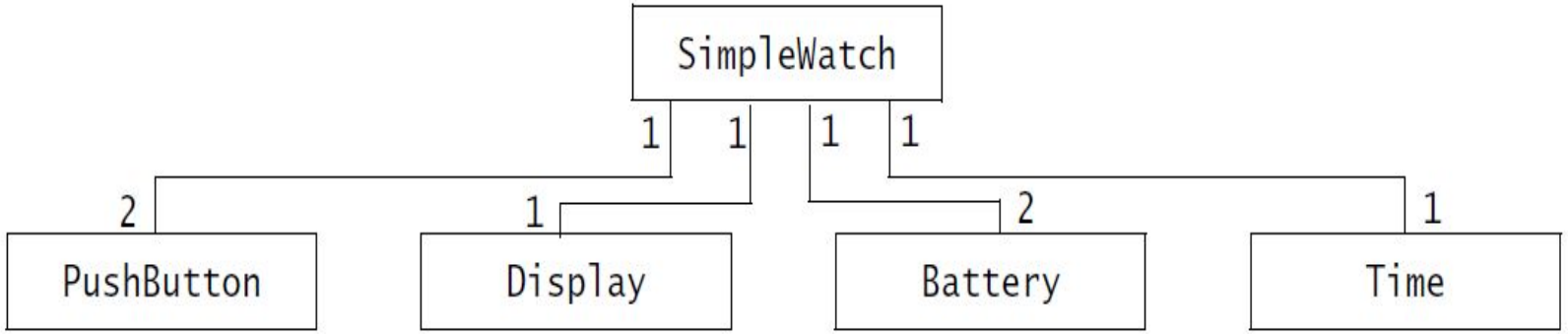
- System development focuses on three different models of the system
  - The **functional model**, represented in UML with use case diagrams
  - The **object model**, represented in UML with class diagrams, describes the structure of the system in terms of objects, attributes, associations, and operations
  - The **dynamic model**, represented in UML with interaction diagrams, state machine diagrams, and activity diagrams, describes the internal behavior of the system

Use Case Diagrams

- Use cases are used during *requirements elicitation* and *analysis* to represent the functionality of the system
- Use cases focus on the behavior of the system from an external point of view
- A use case *describes a function provided by the system* that yields a visible result for an actor
- The identification of actors and use cases results in the *definition of the boundary of the system*, that is, in differentiating the tasks accomplished by the system and the tasks accomplished by its environment
- The *actors are outside the boundary of the system*, whereas the *use cases are inside the boundary of the system*



Class Diagrams

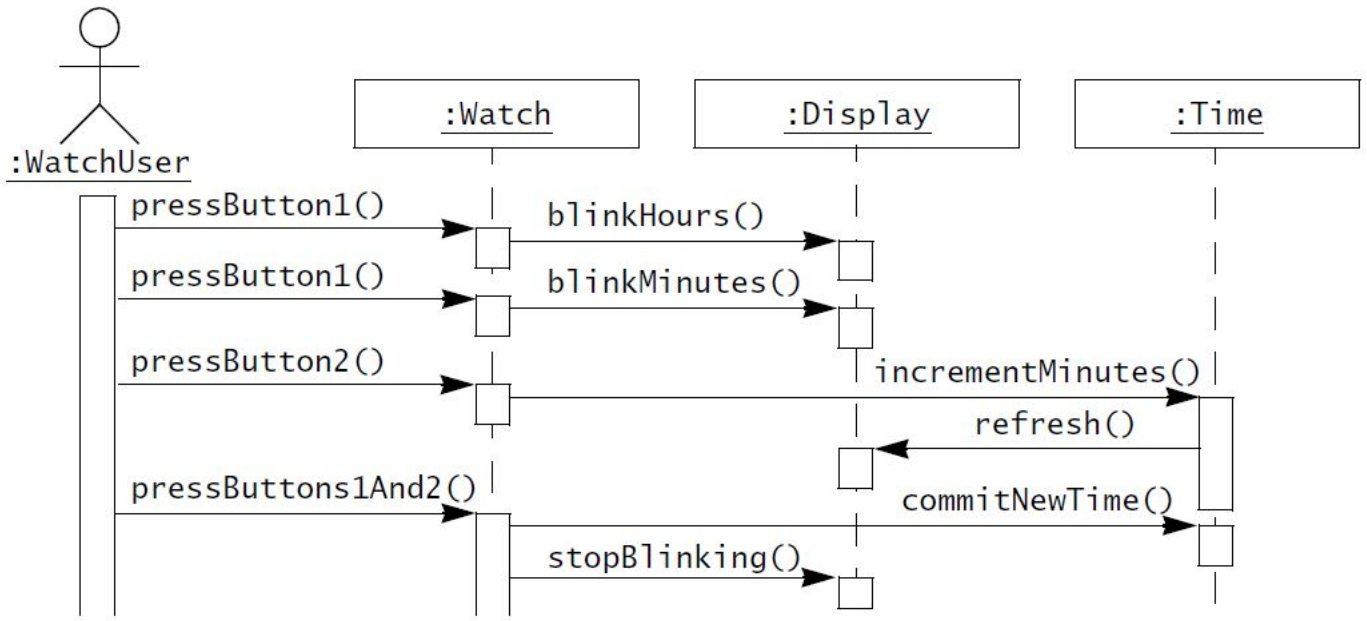


- Class diagrams are used to *describe the structure of the system*.
- Classes are abstractions that specify the common structure and behavior of a set of objects
- **Objects** are instances of classes that are *created, modified, and destroyed during the execution of the system*
- An object has state that includes the values of its attributes and its links with other objects
- Class diagrams describe the system in terms of objects, classes, attributes, operations, and their associations

Interaction Diagrams

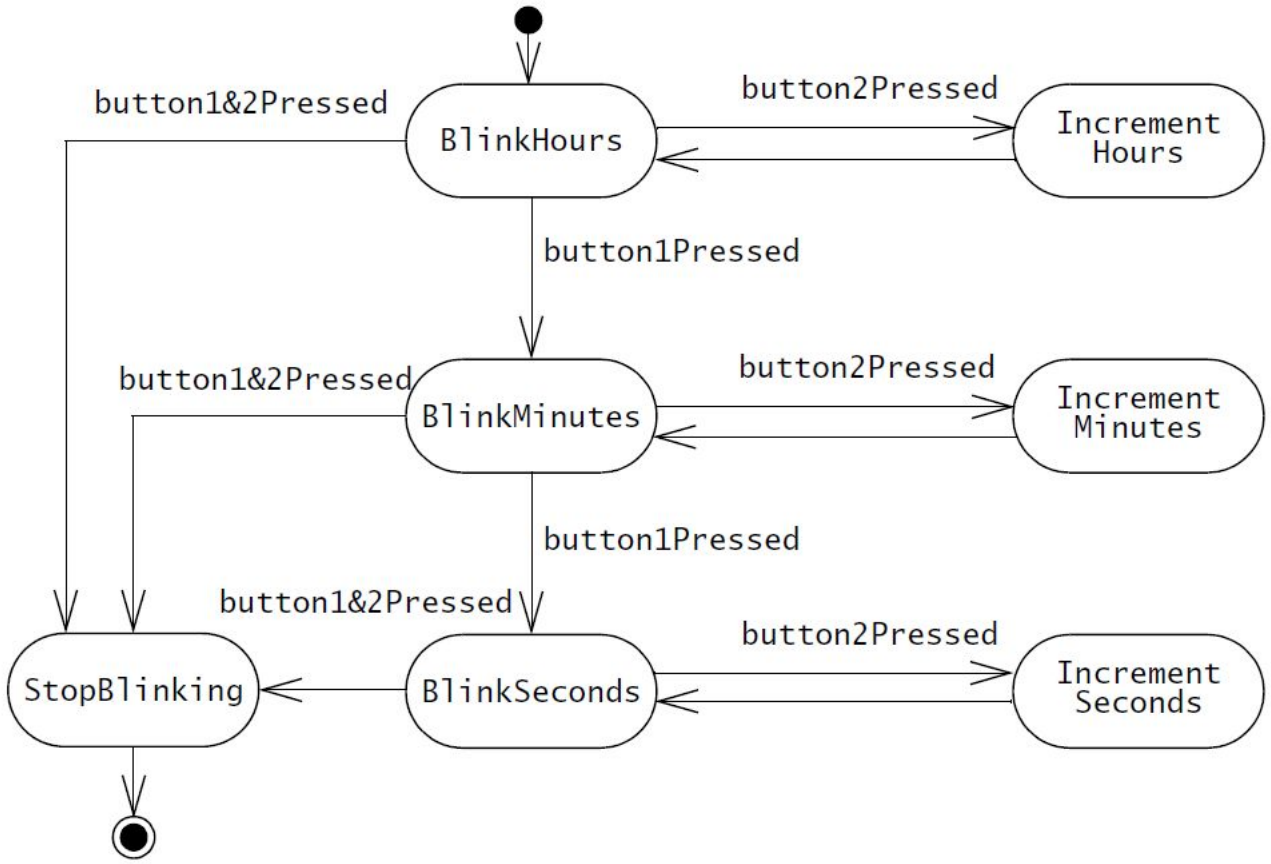
Seq D

- Interaction diagrams are used to formalize the **dynamic behavior** of the system and to *visualize the communication among objects*
- They are useful for identifying additional objects that participate in the use cases
- An interaction diagram represents the interactions that take place among these objects
- Object names are <u>underlined</u> to denote that they are *instances*



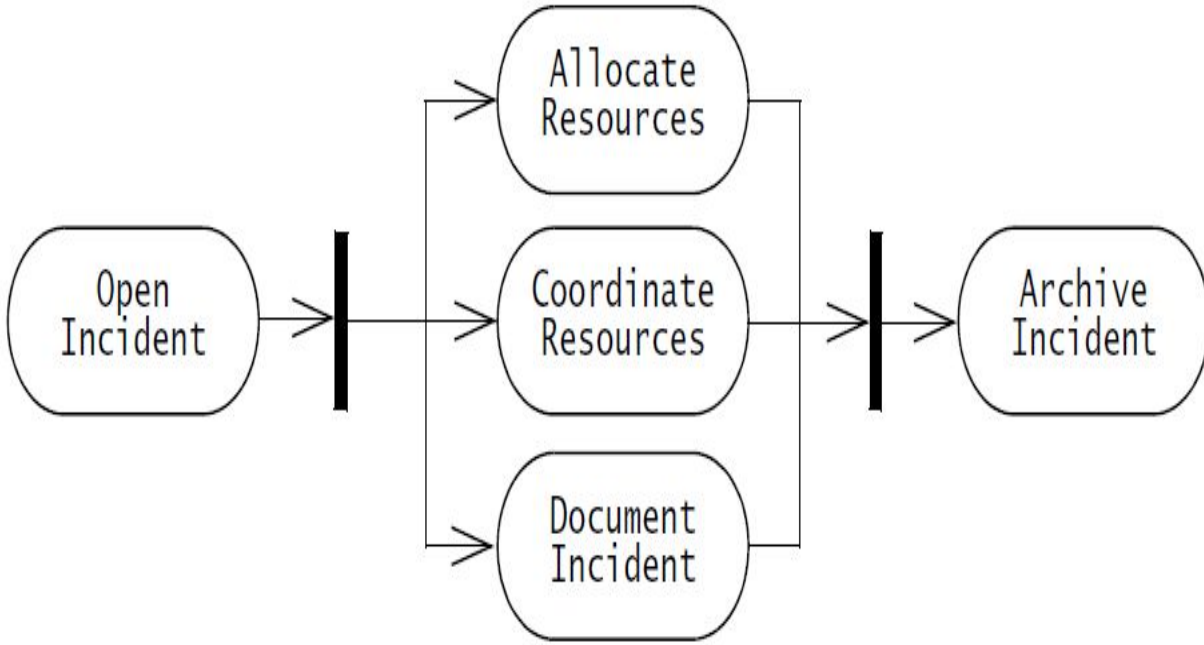
State Machine Diagrams

- State machine diagrams describe the **dynamic behavior** of an *individual object* as a number of states and transitions between these states
- A state represents a particular set of values for an object
- Given a state, a transition represents a future state the object can move to and the conditions associated with the change of state
- The state machine diagram *focuses on the transitions between states* as a result of external events for an individual object



Activity Diagrams

- An activity diagram describes the *behavior of a system in terms of activities*
- Activities are modeling elements that *represent the execution of a set of operations*
- The execution of an activity can be triggered by the completion of other activities, by the availability of objects, or by external events
- Activity diagrams are similar to flowchart diagrams in that they can be used to represent control flow (i.e., the order in which operations occur) and data flow (i.e., the objects that are exchanged among operations)



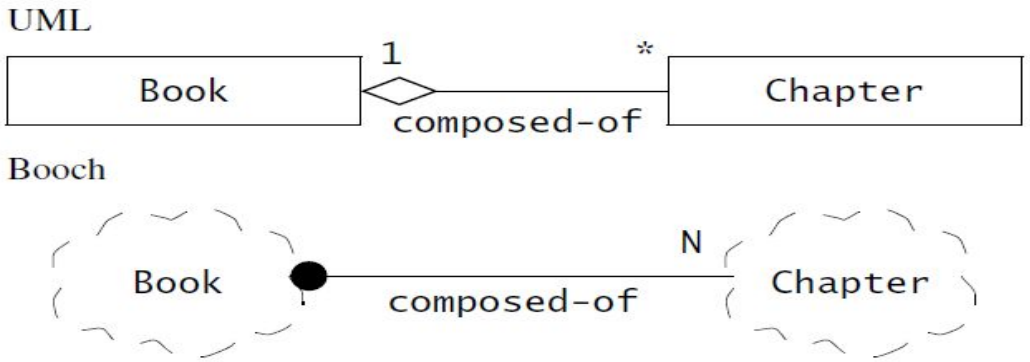
Modeling Concepts

- An engineering **system** is an organized set of communicating parts
- It is designed for a specific purpose. For example
  - A car, composed of four wheels, a chassis, a body, and an engine, is designed to transport people.
  - A watch, composed of a battery, a circuit, wheels, and hands, is designed to measure time.
  - A payroll system, composed of a mainframe computer, printers, disks, software, and the payroll staff, is designed to issue salary checks for employees of a company.
- Parts of a system can in turn be considered as simpler systems called **subsystems**
  - The engine of a car, composed of cylinders, pistons, an injection module, and many other parts, is a subsystem of the car
- This subsystem decomposition can be **recursively** applied to subsystems
- **Objects** represent the end of this recursion, when each piece is simple enough that we can fully comprehend it without further decomposition

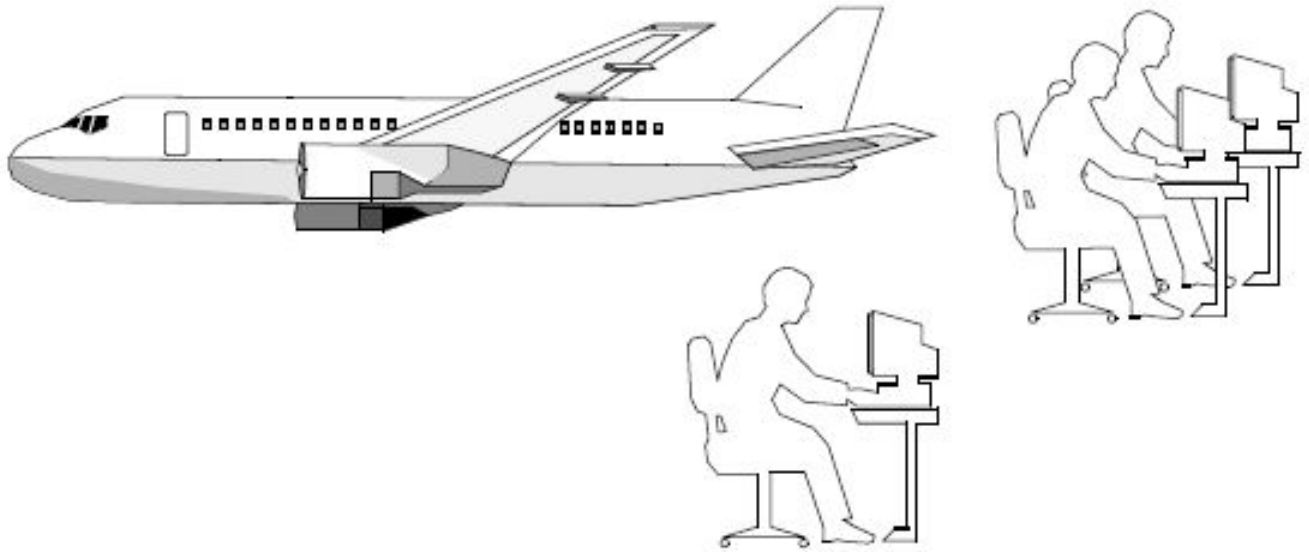
Modeling

- Many systems are made of numerous subsystems interconnected in complicated ways, often so complex that no single developer can manage its entirety
- **Modeling** is a means for *dealing with this complexity*
- **Complex systems** are generally described by more than one model, each focusing on a different aspect or level of accuracy
- **Modeling** means *constructing an abstraction of a system* that focuses on interesting aspects and ignores irrelevant details
- Modeling allows us to deal with complexity through a ***divide-and-conquer approach***: For each type of problem we want to solve, we build a model that only focuses on the issues relevant to the problem
- Generally, modeling focuses on building a model that is *simple enough for a person to grasp completely*
- A rule of thumb is that each entity should contain at most **7 ± 2 parts** [Miller, 1956]
- The set of all models built during development is called the **system mode**
- A **view** focuses on a subset of a model to make it understandable

Notations



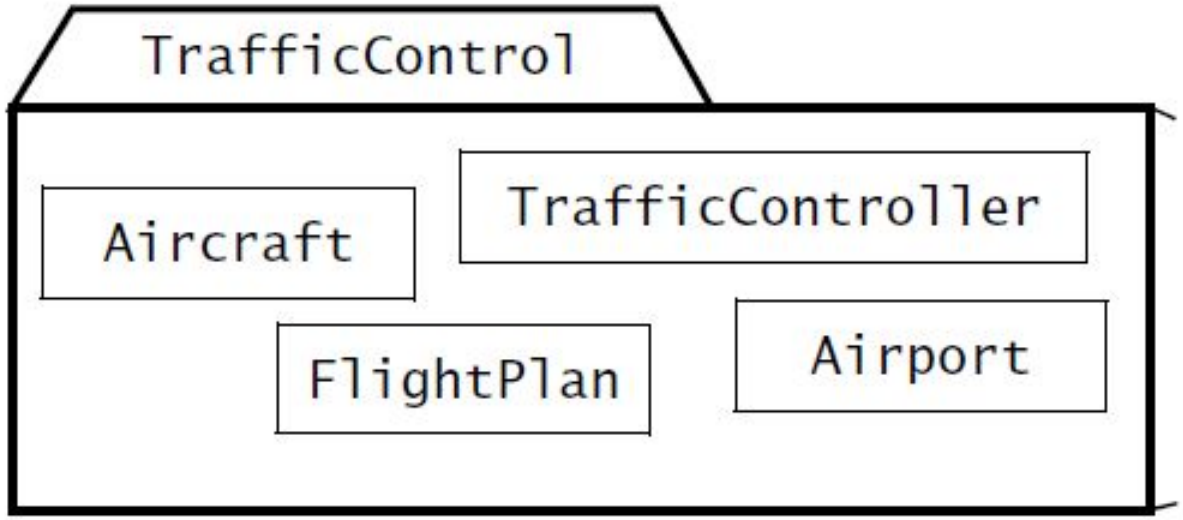
- **Notations** are graphical or textual rules for representing views
- A UML class diagram is a graphical view of the object model
- In UML class diagrams, a **rectangle** with a title represents a *class*
- A **line** between two rectangles represents a *relationship* between the two corresponding classes
- Note that different notations can be used to represent the same view
- In software engineering, there are many other notations for modeling systems
  - **UML** describes a system in terms of classes, events, states, interactions, and activities
  - **Data flow diagrams** [De Marco, 1978] depict how data is retrieved, processed, and stored
  - **Z Schemes** [Spivey, 1992] represent the system in terms of invariants (conditions that never change) and in terms of what is true before and after the execution of an operation.
- Each notation is tailored for a different problem.



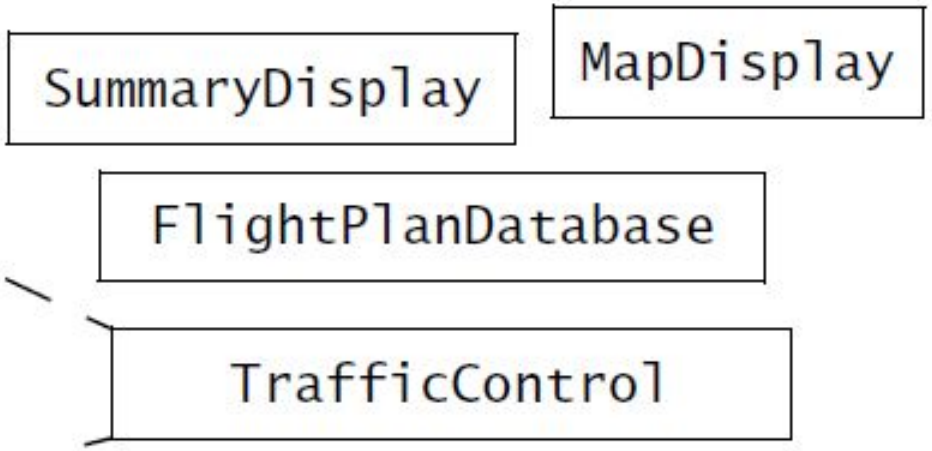
Application Domain

Solution Domain

Application Domain Model

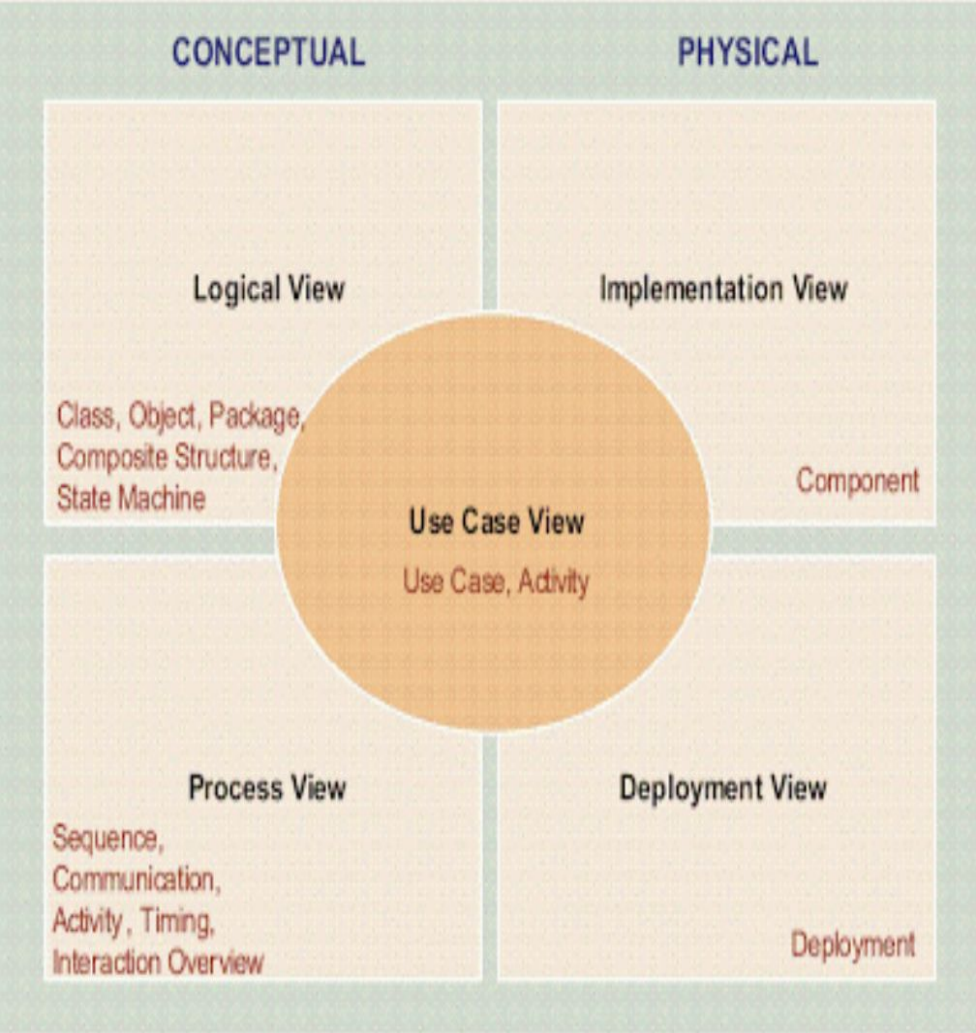


System Model



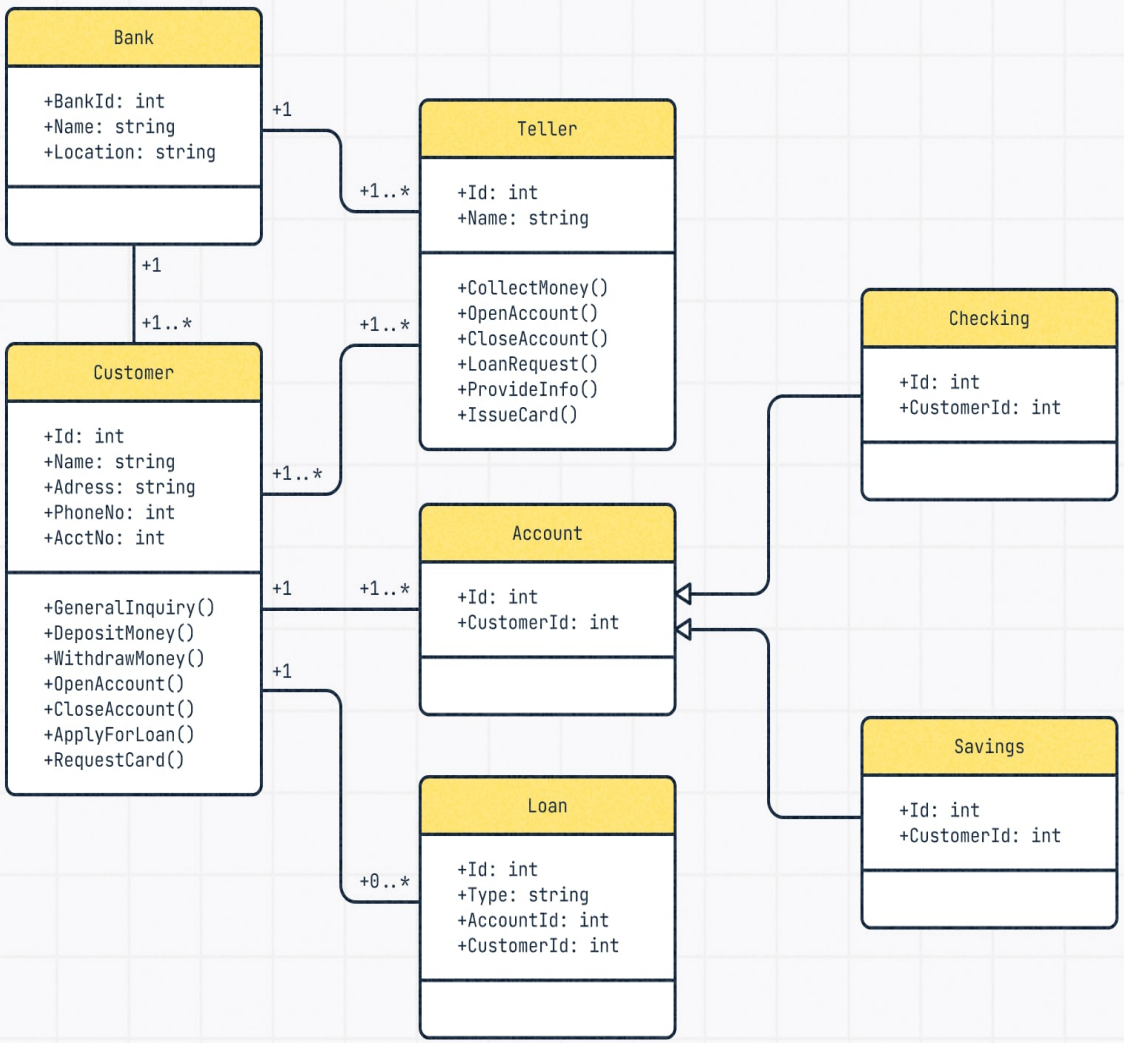
4 + 1 Views System Architecture - UML

- In the realm of software engineering and system design, creating a comprehensive and well-structured architecture is essential for building complex systems
- Unified Modeling Language (UML) is a powerful tool that facilitates the visualization and documentation of system architectures
- One widely adopted approach for modeling system architectures using UML is the "4 + 1" view model
- This model offers a holistic perspective on system architecture, breaking it down into five distinct views that collectively provide a comprehensive understanding of the system



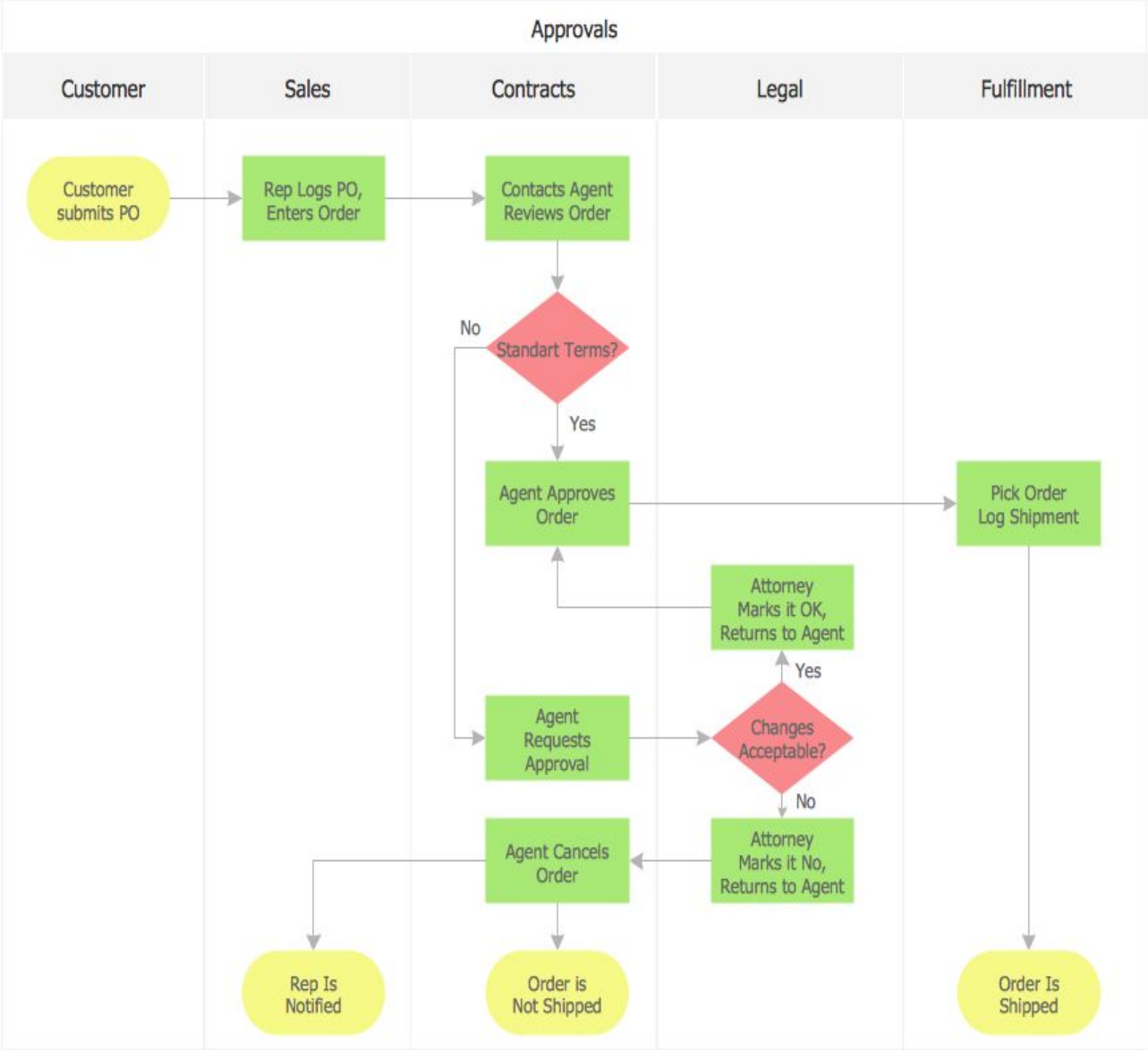
1. **Logical View**

- The Logical View, often referred to as the "**Class Diagram**," is the first of the "4 + 1" views
- It primarily focuses on the *static structure* of the system
- This view deals with the essential components of the system, such as classes, objects, relationships, and their attributes
- Class diagrams visually represent the classes and their associations in the system, helping to understand how various components interact with one another
- This view provides insights into the overall design and organization of the system's functionality



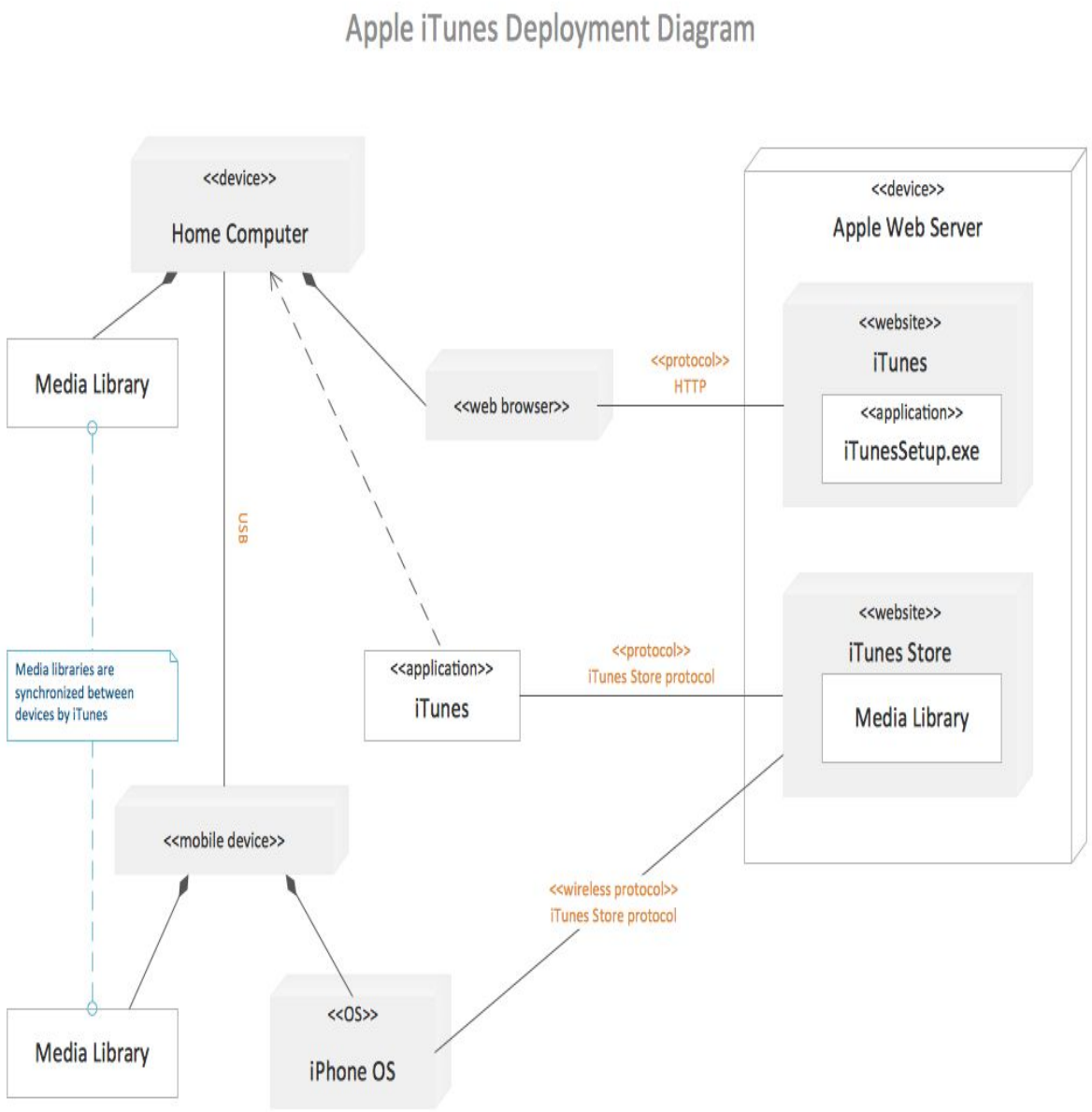
2. Process View

- The Process View, also known as the "**Activity Diagram**," delves into the dynamic aspects of the system
- It describes the system's behavior and the flow of control between various activities or processes
- Activity diagrams are particularly helpful in illustrating how different components of the system collaborate to accomplish specific tasks or use cases
- This view is crucial for understanding the sequence of actions and interactions in the system, making it an essential tool for system designers and developers



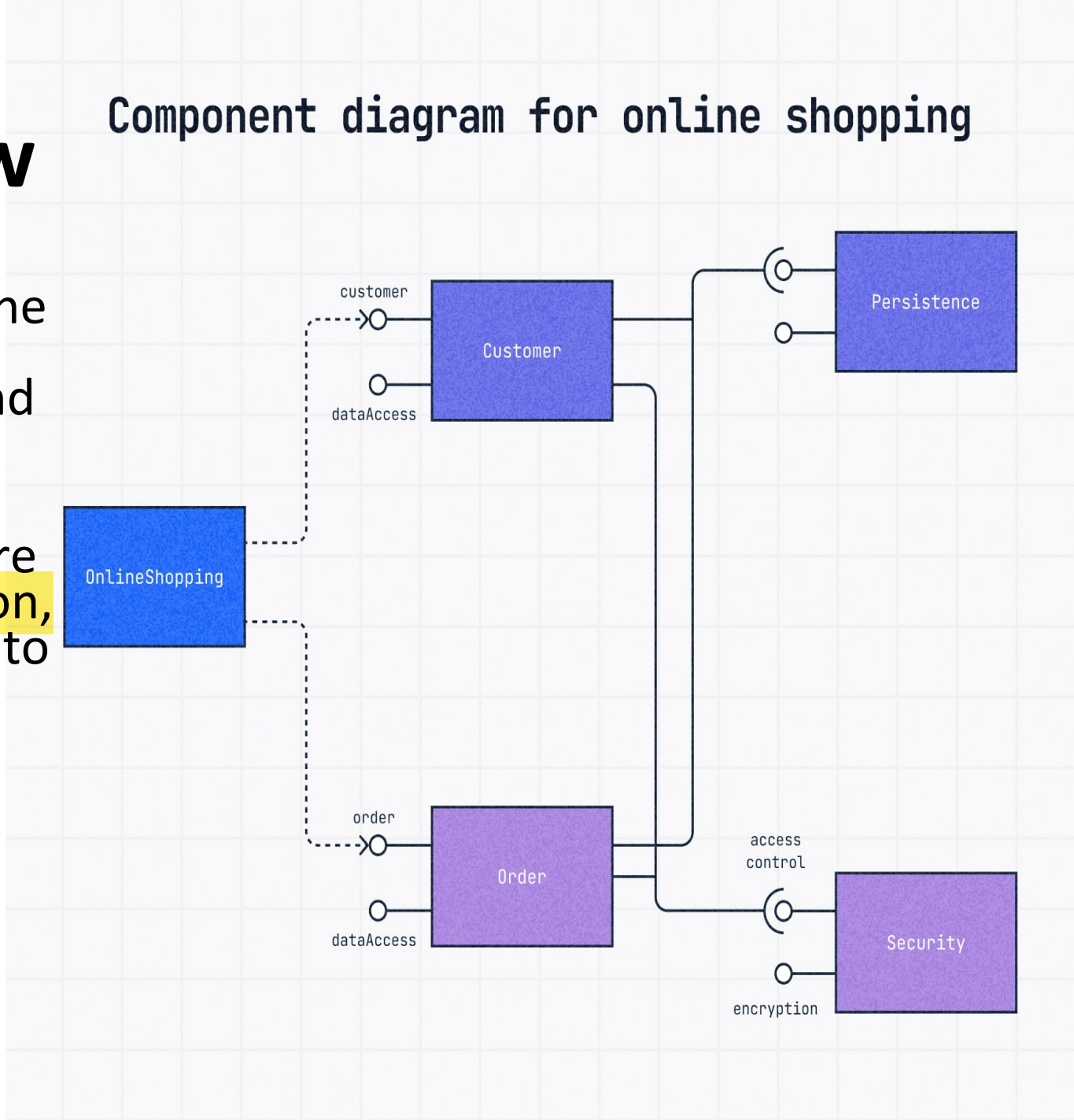
3. **Physical View**

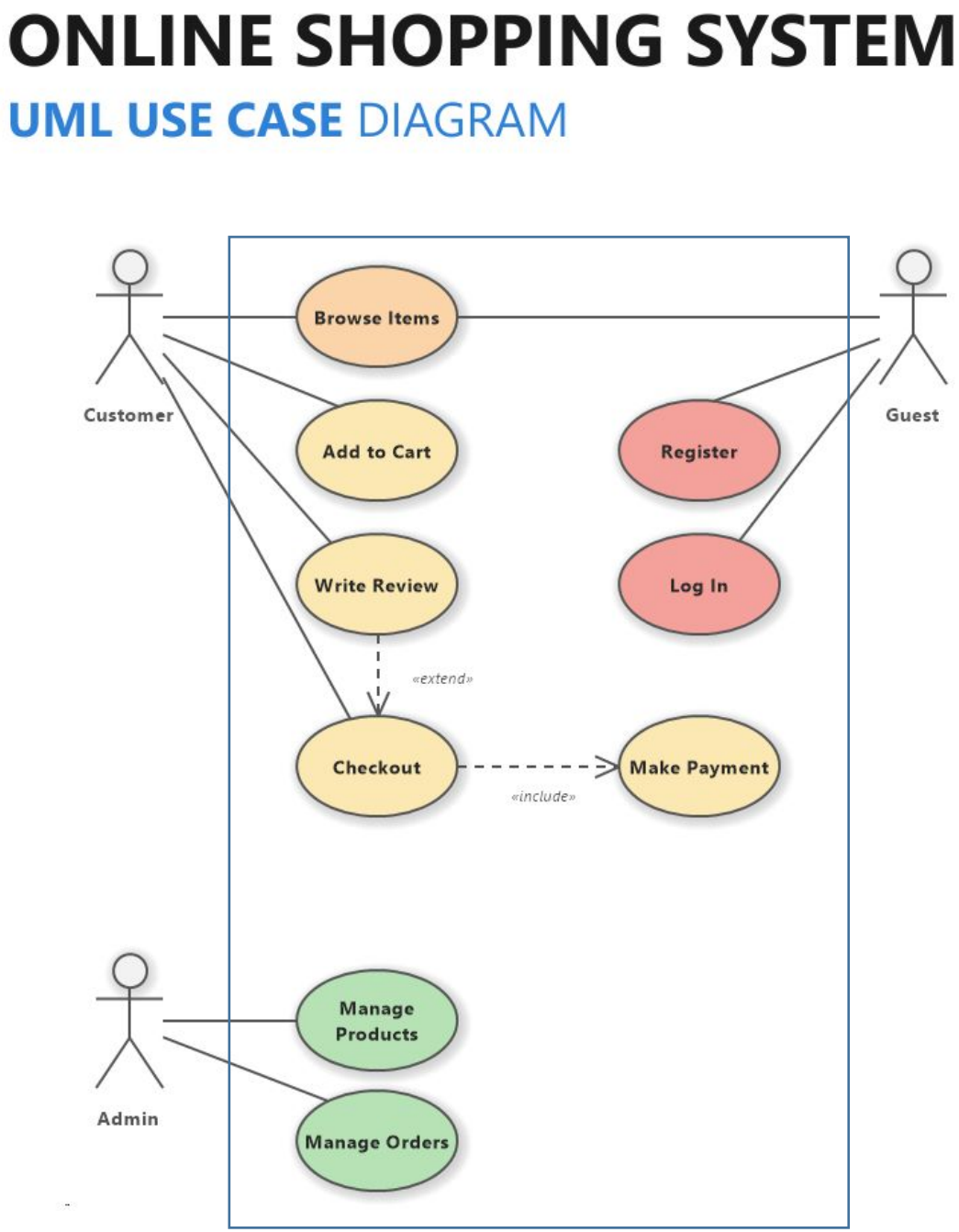
- The Physical View, represented by the "**Deployment Diagram,**" provides insights into how the system's software components are deployed on hardware infrastructure
- It includes details about servers, nodes, and their interconnections
- Deployment diagrams help in addressing questions related to scalability, fault tolerance, and resource allocation
- This view ensures that the system architecture aligns with the physical constraints and requirements of the deployment environment



4. **Development View**

- The Development View, also known as the "**Component Diagram**," focuses on the organization of software components and their relationships within the development environment
- It is instrumental in mapping the software architecture to the actual implementation, making it easier for development teams to understand how different modules are distributed across the codebase
- This view aids in managing the development process, ensuring that software components are developed, tested, and integrated efficiently.



5. **Use Case View**

- The fifth view in the “4 + 1” model is the Use Case View
- While the other four views primarily concentrate on the internal aspects of the system, the Use Case View emphasizes the system’s functionality from the user’s perspective
- Use case diagrams depict how users interact with the system, showcasing various scenarios and the corresponding system responses
- This view helps in defining and validating system requirements, ensuring that the architecture aligns with the end-users’ needs and expectations